Ecole Supérieure d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

Université Internationale de Rabat (UIR)
École Supérieure d'Informatique et du Numérique
Major: Cybersecurity

4th Year – Cybersecurity Engineering

Summer Internship Report – July-August 2026

Implementation of a SIEM System with an AI/ML-Enhanced Detection Layer for Security Event Monitoring and Intelligent Log Analysis

Hosted at:

YaneCode Digital

Prepared by:
Wissal EL BEKKALI

Supervised at YaneCode by:
M. Jamal ET-TOUSY

Internship Period: from 01/07/2026 to 31/08/2026

Academic Year: 2025/2026

Acknowledgments

I would like to express my sincere thanks to everyone who contributed to this project and supported me throughout my internship.

- To my professional supervisor, M. Jamal ET-TOUSY, for his rigorous guidance, constant availability, and the sound technical advice that guided me step by step through the world of cybersecurity.
- To the entire YaneCode Digital team, for their warm welcome and for providing a stimulating and supportive learning environment.
- To the faculty of Université Internationale de Rabat (UIR), for the quality of the education provided throughout my studies, which gave me the solid foundation needed to carry out this project.
- To my family and those close to me, for their unconditional support and constant encouragement throughout this work.

Résumé

Face à la croissance des cybermenaces, ce rapport présente la conception d'un SIEM basé sur Wazuh chez YaneCode Digital, enrichi d'une couche d'analyse par intelligence artificielle. Une architecture complète a été déployée sur un laboratoire de test, avec quatre règles Sigma validées en conditions réelles (force brute, élévation de privilèges, scan réseau, exfiltration). Une couche de machine learning a ensuite été explorée en plusieurs itérations – Isolation Forest, puis Random Forest et XGBoost sur le dataset fusionné – échouant d'abord dans les deux sens sur du trafic réel (faux négatifs ou faux positifs massifs), pour une cause identifiée : la classification connexion par connexion ne peut pas capter un pattern de scan, qui se définit par la répétition. L'ajout de features temporelles agrégées par IP source (fenêtre glissante de 10 secondes) a résolu ce problème structurel : le modèle Random Forest final, validé sur un scan totalement inédit capturé après l'entraînement puis ré-entraîné et re-validé directement sur le serveur SIEM après correction d'un bug d'étiquetage, atteint 99,78% de recall pour 0,06% de faux positifs (résultat officiel final). Ce modèle couvre uniquement le **scan réseau** ; la force brute, l'élévation de privilèges et l'exfiltration restent détectées exclusivement par les règles déterministes Wazuh/Sigma. Une interface de prédiction (Streamlit), indépendante de Wazuh, a ensuite été développée et déployée sur le serveur du laboratoire pour exposer ce modèle en complément de la détection déterministe. **Le moteur déterministe Wazuh/Sigma est validé et opérationnel sur le lab ; la couche machine learning constitue un prototype validé et déployé en laboratoire, non encore intégré nativement à Wazuh.**

Mots-clés : SIEM, Cybersécurité, Wazuh, Règles Sigma, Intelligence Artificielle, Machine Learning, Détection d'anomalies.

Abstract

In the face of growing cyber threats, this report presents the design of a Wazuh-based SIEM at YaneCode Digital, enhanced with an AI-driven analysis layer. A complete architecture was deployed on a test lab, with four Sigma detection rules validated under real attack conditions (brute force, privilege escalation, network scan, data exfiltration). A machine-learning layer was then explored through several iterations – Isolation Forest, then Random Forest and XGBoost on the merged dataset – initially failing in both directions against live traffic (false negatives or massive false positives), a failure eventually traced to a structural cause: per-connection classification cannot capture a scan pattern, which is defined by repetition rather than by any single connection. Adding behavioural aggregate features over a sliding time window per source IP resolved this: the final Random Forest model, validated on a genuinely unseen scan captured after training and re-validated on the lab's SIEM server after a labelling-bug fix, reaches 99.78% recall at 0.06% false positives (official final result). This model covers **network scan only**; brute force, privilege escalation, and data exfiltration remain detected exclusively by the deterministic Wazuh/Sigma rules. A prediction interface (Streamlit), independent of Wazuh, was then built and deployed on the lab server to expose this model alongside the existing deterministic detection. **The deterministic Wazuh/Sigma engine is validated and operational on the lab; the machine-learning layer is a prototype, validated and deployed in the lab, not yet natively integrated into Wazuh.**

Keywords: SIEM, Cybersecurity, Wazuh, Sigma Rules, Artificial Intelligence, Machine Learning, Anomaly Detection.

List of Figures

1.1 YaneCode Digital logo 2
1.2 YaneCode Robotic and YaneCode Academy logos 2
1.3 YaneCode Intern and YaneCode Gaming logos 2
1.4 YaneCode organization chart 3

2.1 Gantt chart of the internship project (July–August 2026) 10

3.1 Technical architecture of the Wazuh SIEM system 12
3.2 Wazuh Dashboard overview, 20 July 2026 14
3.3 Wazuh Indexer, Suricata events, 19 July 2026 15
3.4 Wazuh Indexer, Zeek events (rule 100100), 20 July 2026 16
3.5 End-to-end dataset and model pipeline (second attempt) 17
3.6 Confusion matrices, v4 models, 20 August 2026 scan 23
3.7 Terminal output, v4 fresh-scan validation script 24
3.8 Streamlit interface, manual-entry mode 30
3.9 Streamlit interface, CSV batch-upload mode 31
3.10 Terminal output, multi-session evaluation script 31
3.11 Terminal output, memory benchmark 32
3.12 Streamlit interface after a cold restart 32
3.13 Training scripts and MODEL_VERSIONS.md on GitHub 33

List of Tables

2.1 Comparison of open-source SIEM solutions 7

3.1 Collection sources deployed and validated 14
3.2 Detection rules and live-attack validation 15
3.3 Isolation Forest final evaluation metrics (iteration 5) 16
3.4 Class distribution across the raw CICIDS2017 corpus, before and after removing exact duplicates 18
3.5 Baseline classifier results on the merged dataset 18
3.6 Isolated dataset experiments 19
3.7 Live-traffic validation results for both deployed models 21
3.8 v4 (temporal features) validation on a genuinely out-of-sample scan – full metrics . 23
3.9 Per-session evaluation of the deployed v4 Random Forest model 26
3.10 v4 Random Forest – throughput and latency 27
3.11 v4 Random Forest – memory footprint 27

List of Abbreviations

API	Application Programming Interface
HTTPS	HyperText Transfer Protocol Secure
IA / AI	Intelligence Artificielle / Artificial Intelligence
IDS	Intrusion Detection System
IoT	Internet of Things
JSON	JavaScript Object Notation
ML	Machine Learning
PCAP	Packet Capture
RDP	Remote Desktop Protocol
RF	Random Forest
SID	Signature/Rule Identifier (Wazuh)
SIEM	Security Information and Event Management
SMOTE	Synthetic Minority Oversampling Technique
SOC	Security Operations Center
TCP / UDP	Transmission Control Protocol / User Datagram Protocol
TLS	Transport Layer Security
UIR	Université Internationale de Rabat
VM	Virtual Machine
XGBoost	Extreme Gradient Boosting
XML	Extensible Markup Language
YAML	YAML Ain’t Markup Language

Table of Contents

Acknowledgments **i**

Résumé **ii**

Abstract **iii**

List of Abbreviations **vi**

General Introduction **1**

1 Presentation of the Host Organization **2**
1.1 Introduction 2
1.2 Presentation of the Company 2
1.3 YaneCode Organization Chart 3
1.4 Presentation of Services 4
1.5 Main Projects 4
1.6 Conclusion 5

2 Presentation of the Internship Project **6**
2.1 Introduction 6
2.2 Terms of Reference 6
2.3 Tools and Technical Environments 6
2.4 Agile Methodology 8
2.5 Project Management Approach 9
2.6 Gantt Chart 9
2.7 Conclusion 9

3 Project Realization **11**
3.1 Introduction 11
3.2 Design 11
3.2.1 Enterprise Architecture (YaneCode) 11
3.2.2 Technical Architecture of the SIEM System (Wazuh) 11
3.2.3 AI/ML Layer Design 11
3.3 Implementation 13
3.3.1 Test Lab Deployment 13
3.3.2 Detection Engineering 14
3.3.3 AI/ML Detection Layer: Iterations and Diagnosis 15
3.3.4 Deployment and Live-Traffic Validation 20
3.3.5 Prediction Interface and Lab-Server Deployment 24
3.3.6 Multi-Session, Multi-Attacker Validation 25

3.3.7 Operational Performance Evaluation 26
3.4 Conclusion 28

General Conclusion **34**

Appendices **36**

Bibliography **38**

General Introduction

Presentation of the subject. The digital transformation of businesses inevitably comes with an expanding attack surface. Faced with increasingly sophisticated threats, traditional perimeter-protection methods are no longer sufficient, and organizations need full visibility over their systems to detect abnormal behaviour before it causes irreversible damage. It is within this proactive dynamic that my internship project at YaneCode Digital takes place: implementing a Security Information and Event Management (SIEM) system that centralizes, correlates, and alerts on suspicious activity across the corporate network in real time, complemented by an additional layer of intelligent analysis powered by artificial intelligence.

Problem statement. YaneCode Digital's activity spans several heterogeneous, internet-facing systems – an e-commerce platform, IoT infrastructure, an academy server, and general enterprise services – each generating its own logs with no centralized visibility. The central question this internship addresses is: how can YaneCode Digital gain unified, real-time visibility over malicious behaviour across its heterogeneous server fleet, when no centralized log-monitoring solution currently exists? Answering this required both a deterministic layer – rules that reliably catch known attack patterns – and an exploratory layer – machine learning that could, in principle, catch patterns no rule was written for.

Methods and technologies. The deterministic core was built on Wazuh, an open-source SIEM platform natively compatible with the Sigma rule format, deployed on a four-VM test lab (Ubuntu Server, Windows 11, Kali Linux, Apache) and fed by Suricata, Zeek, Syslog, and Winlogbeat. The exploratory layer combined the public CICIDS2017 dataset [1] with the lab's own traffic to train and evaluate Isolation Forest [2], Random Forest [4], and XGBoost [5] models.

Objectives and report outline. The project pursued two objectives: deliver a validated, operational deterministic detection layer covering four attack categories (brute force, privilege escalation, network scan, data exfiltration), and investigate whether a machine-learning layer could reliably enrich that detection. This report is organized in three chapters: Chapter 1 presents the host organization; Chapter 2 covers the project's context, specifications, tools, and methodology; and Chapter 3 details the full realization of the project – design, implementation, experimentation, and results – followed by a general conclusion on what is validated today and what remains to be done.

CHAPTER 1: Presentation of the Host Organization

1.1. Introduction

This chapter presents the host organization, YaneCode Digital. It covers the company's organizational structure, its specialized branches, its flagship projects, and the broad range of digital services it offers its partners.

1.2. Presentation of the Company

YaneCode Digital is a digital-transformation and AI engineering company based in Safi, Morocco. Its core business is designing tailor-made software and AI-driven solutions for businesses across Morocco and Africa, and it counts over 50 completed projects and 35 clients across 8 sectors among its references, including OCP, Afriquia, COPAG, and ENSAM.



Figure 1.1: YaneCode Digital logo

Since its founding, YaneCode Digital's ecosystem has expanded to include four additional branches: YaneCode Robotic, YaneCode Academy, YaneCode Intern, and YaneCode Gaming.





Figure 1.2: YaneCode Robotic and YaneCode Academy logos





Figure 1.3: YaneCode Intern and YaneCode Gaming logos

- **YaneCode Digital** – The parent agency dedicated to application development, AI/ML, data engineering, and cloud/DevOps for enterprise clients.

- **YaneCode Academy** – A training centre offering programs in web development, data science, AI, and DevOps for students and professionals.
- **YaneCode Robotic** – Industrial robotics, embedded systems, and intelligent automation solutions for Industry 4.0.
- **YaneCode Intern** – An internal platform for managing internships, applications, and intern tracking from onboarding to completion.
- **YaneCode Gaming** – Development of video games, immersive experiences, and serious games for learning and entertainment.

1.3. YaneCode Organization Chart

The company's organization follows a clear structure that favours synergy between operational departments, under the direction of General Management and the General Secretariat: the Logistics Department, the Human Resources Department, the Technical Department, and the Commercial Department.

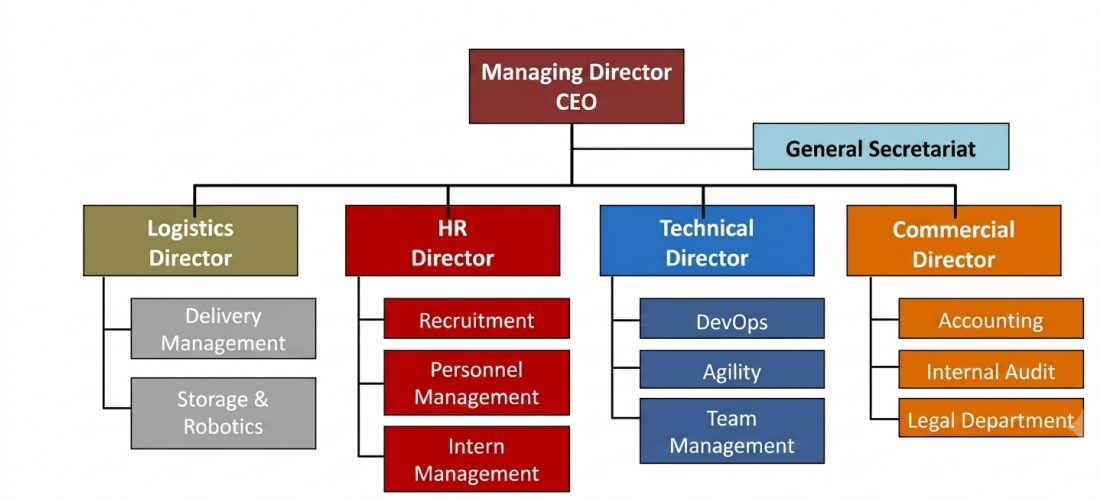


Figure 1.4: YaneCode organization chart

1.4. Presentation of Services

YaneCode Digital deploys a broad range of expertise to meet current technological demands:

- **Web, Mobile & Desktop Development** – Design of high-performance solutions tailored to client requirements, from mock-up to production.
- **Artificial Intelligence & Generative AI** – Predictive models, NLP, computer vision, and adapted LLM solutions.
- **Data Engineering & Automation** – Data pipelines, ETL, Power BI, and intelligent data exploitation.
- **Cloud & Digital Infrastructure** – Cloud architecture, DevOps, CI/CD, and scalable infrastructure.
- **Internet of Things (IoT)** – Connected sensors, edge computing, and industrial IoT solutions.
- **Custom Business Solutions** – ERP, CRM, portals, and business tools tailored to the organization.

1.5. Main Projects

Beyond client engineering work, YaneCode Digital maintains an applied R&D track with published results in international journals and conferences (5+ publications, 3 journals, 4 conferences, one ongoing PhD). Its flagship projects illustrate this AI-first approach:

- **Azul AI** (azulaimaroc.com) – An AI-powered tourism platform offering a personalised discovery experience of Morocco; winner of the Prix Maroc Jeunesse 2024.
- **Menteva** (menteva.ma) – An AI-driven adaptive learning platform for students, teachers, schools, and parents.
- **Agribot** (agribotmorocco.com) – An intelligent agricultural assistant delivered via WhatsApp, helping Moroccan farmers with AI-based advice in Darija and Arabic.
- **MEDIFICS** – A multimodal medical AI solution combining vision-language models for clinical diagnosis support (published at IEEE ICDS 2024).
- **ConcoursPret** (concourspret.com) – An exam-preparation platform for Morocco's competitive school and professional entrance exams.
- **Wazuh SOC Lab** – A Wazuh-based Security Operations Centre laboratory for intrusion detection and SIEM, directly related to the applied security research this internship project contributes to.

1.6. Conclusion

This first chapter provided context for the internship's professional environment. The diversity of YaneCode's activities – spanning enterprise software, AI R&D, robotics, and training – generates a heterogeneous activity flow that calls for a high level of security oversight. It is within this context that the implementation of a unified security supervision system finds its full justification.

CHAPTER 2: Presentation of the Internship Project

2.1. Introduction

This chapter presents the internship project in full: the general context that motivated it, the terms of reference it had to satisfy, the tools and technologies selected, the Agile methodology adopted, and the project-tracking approach and schedule that structured the eight weeks of work.

2.2. Terms of Reference

YaneCode Digital's activity spans several heterogeneous, internet-facing systems – an e-commerce platform (YaneCode Robotic), IoT infrastructure, an academy server handling student data, and general enterprise services – each generating its own logs, in its own format, with no centralized visibility. As the company's client base and public-facing surface grow, this fragmentation becomes a genuine security liability: an incident on any one of these systems (a brute-force attempt, a scan, an unauthorized privilege change, an abnormal data transfer) can go unnoticed until well after the fact, simply because no one is watching all of them at once, in real time, from a single place.

The main mandate of this project is to design an intelligent log-analysis platform capable of monitoring the company's server fleet and issuing instant alerts in the event of a suspected attack. The detection scope primarily targets four categories of threats:

- **Brute-force attacks** – Massive, repeated authentication attempts against administration ports.
- **Port scanning** – Unauthorized discovery of open services on the infrastructure by external entities.
- **Privilege escalation** – Unauthorized attempts to gain administrative control over production servers.
- **Data exfiltration** – Abnormal or large-volume data transfers toward suspicious external destinations.

YaneCode Digital additionally expects, at the end of the internship: a complete folder containing all necessary files and source code of the SIEM platform, a technical report describing each step taken, and a validated, documented detection layer covering the four categories above.

2.3. Tools and Technical Environments

A comparative study of open-source SIEM platforms was conducted, covering Wazuh, Elastic Security, Graylog, and Security Onion. The comparison considered licensing model, ease of deployment, detection-rule ecosystem, and community support.

Technology decision: Wazuh was selected as the platform for the project on the basis of being fully open-source, having a mature rule-correlation engine natively compatible with the

Table 2.1: Comparison of open-source SIEM solutions

Criteria	**Wazuh**	**Elastic Security**	**Graylog**	**Security Onion**
License	100% open-source	Freemium	Open-source + paid	Free, open-source
Install complexity	Moderate/Easy	Medium	Medium	Complex
Native agents	Wazuh Agent	Elastic Agent	Sidecar + Beats	Zeek/Suricata
Minimum RAM	8 GB	16 GB	8 GB	16 GB
Sigma rules	Native	Supported	Partial	Supported

Sigma rule format, and integrating natively with OpenSearch for indexing and a Kibana-style dashboard, without additional licensing constraints. A related decision was made during the architecture study: the original plan to use a standalone Elasticsearch/Kibana stack was dropped in favour of Wazuh's own Indexer and Dashboard components, avoiding a duplicate, harder-to-maintain stack.

A dedicated research phase was spent studying the Sigma rule format and structure: its YAML schema, field-mapping conventions, and log-source taxonomy. Sigma was chosen as the formalism for writing detection logic because it is platform-agnostic – a rule written once can, in principle, be converted to the native query language of several SIEM backends, which protects the investment made in detection engineering even if the underlying platform changes in the future. Existing public Sigma rules relevant to the project's four target attack categories were reviewed on the SigmaHQ public repository, and early sketches of their conversion into Wazuh's native correlation-rule XML syntax were produced. This groundwork was later used directly when building the four production detection rules described in Chapter 3.

The complete technical stack retained for the project spans several layers of the pipeline:

- **Detection and correlation:** Wazuh (Manager, Indexer, Dashboard) as the SIEM core, configured with custom Sigma-derived correlation rules.
- **Network monitoring:** Suricata (signature-based network IDS) and Zeek (in-depth protocol analysis and connection logging), both deployed on the same monitoring host to provide dual detection coverage.
- **Log transport:** Syslog for network-equipment and Linux host logs, and Winlogbeat (OSS edition) for Windows Event Log forwarding.
- **Lab infrastructure:** VMware Workstation for virtualization, with Kali Linux as the dedicated attacker machine used to simulate all four target attack categories under controlled, repeatable conditions.
- **Machine-learning pipeline:** Python, with pandas for data wrangling, scikit-learn for Isolation Forest [2] and Random Forest [4], XGBoost [5] for gradient boosting, and imbalanced-learn (SMOTE [3]) for class-imbalance correction; model training was run on Google Colab to

take advantage of free GPU/CPU resources for the larger CICIDS2017-scale [1] experiments.

- **Live-traffic validation:** CICFlowMeter, installed directly on the SIEM server, to regenerate CICFlowMeter-compatible flow statistics from raw packet captures during the deployment-validation phase described in Chapter 3.

Each tool was selected not in isolation but for how it fits into the six-stage pipeline detailed in Chapter 3 (Collection → Transmission → Normalisation & Correlation → Indexing → Visualisation → AI Enrichment), so that the technology choices made here directly support the architecture presented later in this report.

2.4. Agile Methodology

Agile methodology is a modern approach to software development and project management that emphasizes adaptability, collaboration, and iterative progress. It was introduced as an alternative to traditional, linear project-management methodologies such as the Waterfall model, and its primary goal is to deliver valuable, high-quality software in incremental and manageable iterations while responding to changing requirements and feedback throughout the development process.

Several principles guided the way this project's tasks were planned and adjusted week to week:

- **Customer collaboration over contract negotiation** – continuous involvement of the supervisor, with weekly progress reviews used to re-align priorities based on real feedback rather than a fixed upfront plan.
- **Working software over comprehensive documentation** – each week's priority was a working, testable increment of the SIEM (a new agent connected, a new rule validated, a new model trained) rather than documentation produced ahead of the implementation.
- **Responding to change over following a plan** – the project schedule itself was adjusted more than once during the internship, reflecting genuine discoveries rather than a rigid original plan.
- **Incremental and iterative development** – both the detection-rule engineering (Chapter 3) and the machine-learning phase were built rule-by-rule and iteration-by-iteration, each iteration diagnosed before moving to the next.

Among the common Agile frameworks – Scrum (time-boxed sprints with defined roles and ceremonies), Kanban (continuous-flow visual board with work-in-progress limits), and Extreme Programming (engineering practices such as TDD and continuous integration) – **Kanban** was adopted for this project. It was chosen over Scrum because the internship's single-contributor, continuously-evolving nature (a diagnosis in one week directly reshaping the next week's priorities, as happened repeatedly during the machine-learning phase) fit a continuous-flow board better than fixed-length sprints with upfront sprint planning.

2.5. Project Management Approach

The project was tracked using YaneCode's internal Kanban platform (`https://yanecode.org`), where each task was broken down and tracked according to its progress state, moving visually across four columns as work advanced:

- **To Do** – Detailed analysis of the security terms of reference, writing and validation of complex Sigma rules specific to the databases.
- **In Progress** – Configuration of log-collection agents on production target servers and TLS encryption setup.
- **In Review** – Functional validation of dashboards and tuning phases to minimise false positives on brute-force alerts.
- **Done** – Deployment of the central Wazuh Manager instance, completion of the technical state-of-the-art benchmark, creation of the test lab (4 operational VMs).

At the time of project initiation, tasks were divided and the corresponding labels placed in the "To Do" column; as the project progressed, the board visually reflected completion by moving each label through the workflow to "Done". This lightweight tracking proved particularly useful during the machine-learning phase, where new diagnostic tasks (e.g. "isolate CICIDS2017-only experiment", "verify Zeek label gap") were added mid-stream in direct response to the previous week's results, rather than being planned from the start – a pattern the Kanban board accommodates naturally, unlike a fixed sprint backlog.

2.6. Gantt Chart

The figure below presents the project timeline actually followed over the eight weeks of the internship, from the initial SIEM benchmark to the final report. Tasks are colour-coded by nature – the deterministic SIEM build (blue), the AI/ML exploration and diagnosis phase (orange), the validated and deployed AI/ML work (green), and documentation (purple) – with the two progress presentations and the internship end date marked as milestones.

2.7. Conclusion

This chapter framed the project's problem, methodology, and scope, and confirmed the technical and economic soundness of the Wazuh ecosystem. Beyond meeting YaneCode Digital's lightweight infrastructure constraints, adopting the Sigma formalism guarantees the longevity and interoperability of the detection rules implemented. The following chapter details how this plan was executed.

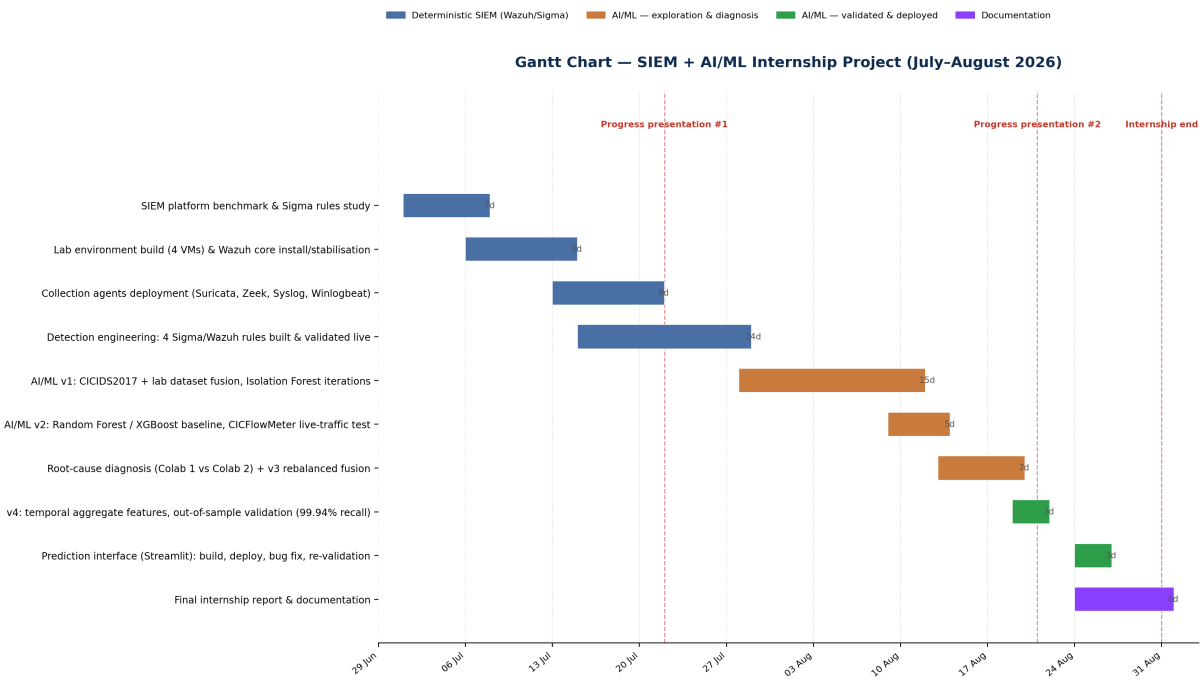


Figure 2.1: Gantt chart of the internship project (July–August 2026)

CHAPTER 3: Project Realization

3.1. Introduction

This chapter presents the full realization of the project. It is organized in two parts, following the actual logic of the work: **Design** (3.2), which covers the architecture and design decisions made before implementation began, and **Implementation** (3.3), which covers the lab deployment, the detection engineering, the machine-learning layer's iterations and diagnosis, and its deployment and validation against live-captured traffic. Each figure is commented in place, and the chapter closes with a single, honest assessment of what is operational today and what is not.

3.2. Design

3.2.1. Enterprise Architecture (YaneCode)

For log centralisation to succeed, the SIEM had to integrate seamlessly into the company's existing architecture:

- **Users & Clients** connect to the company's global application services.
- **E-commerce Server (YaneCode Robotic)** – generates web logs (Apache/Nginx) analysed by the agent.
- **Robotic IoT Server** – reports sensor connectivity data and logs.
- **Academy Server** – records transactions and access via a PostgreSQL database.

3.2.2. Technical Architecture of the SIEM System (Wazuh)

The capture and processing of security alerts follows a highly optimised logical flow, structured into six key stages: **Collection** → **Transmission** → **Normalisation & Correlation** → **Indexing** → **Visualisation** → **AI Enrichment**. Figure 3.1 illustrates this pipeline.

The Wazuh Manager decodes raw data and applies the rule engine to generate alerts in JSON format, which are then pushed to the Indexer (OpenSearch) to be rendered graphically in the Dashboard (SOC), before being passed to the AI layer (machine-learning risk classification and anomaly detection) for final enrichment. As detailed in Section 3.3.3, this last stage is the part of the pipeline still under active development.

3.2.3. AI/ML Layer Design

The machine-learning layer was designed to sit at the end of the existing pipeline, without challenging the technical architecture already deployed: *Raw logs → Sigma rules (detection) → Wazuh Manager (alert) → AI layer (enrichment) → SOC Dashboard.* The design intent was to classify alert risk levels and catch behavioural anomalies that fixed rules might miss, by combining the public CICIDS2017 dataset with the lab's own captured alerts, on the recommendation of the supervisor. The initial architecture sketch (Figure 3.1) considered an LLM-based alert

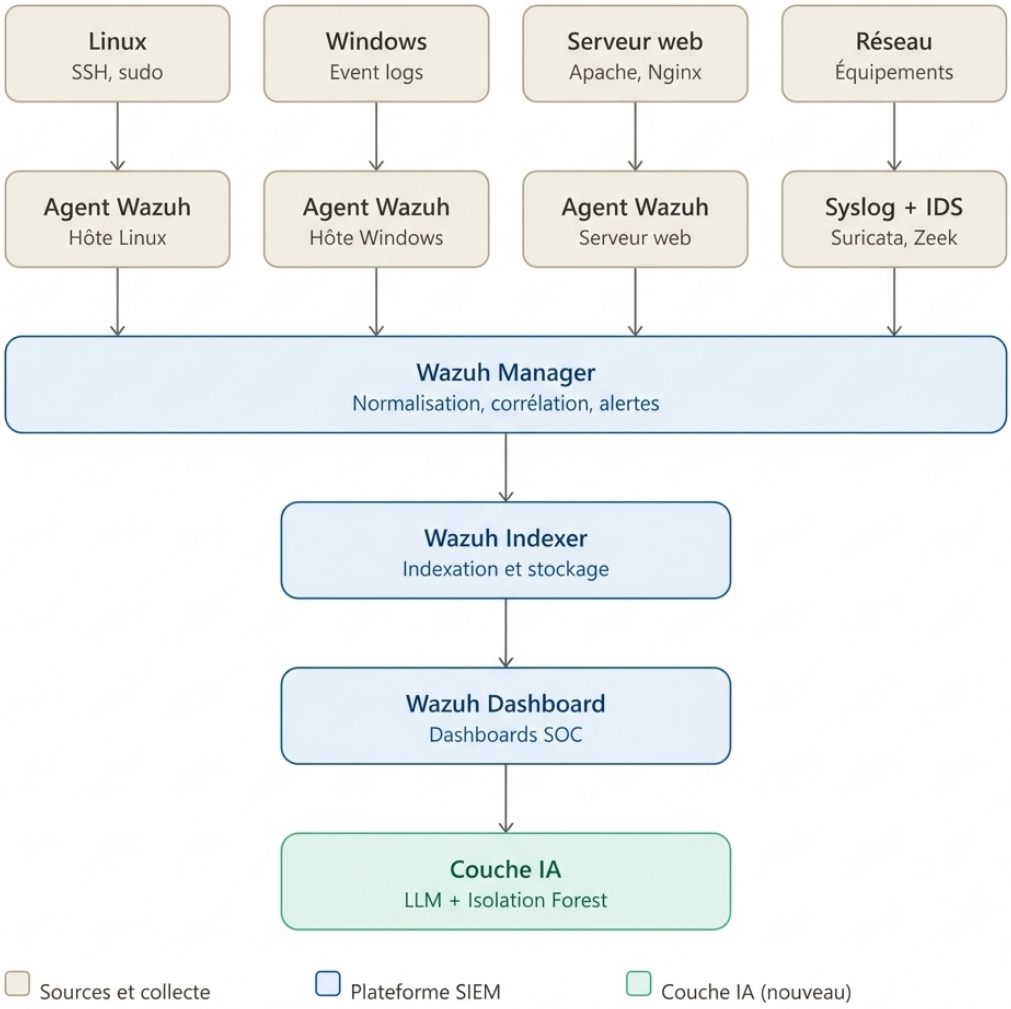


Figure 3.1: Technical architecture of the Wazuh SIEM system

classifier alongside Isolation Forest for this enrichment step; in practice, priority was given to the anomaly-detection and supervised-classification path described below, which the supervisor identified as the more immediately valuable direction given the project's time constraints, and LLM-based classification was not pursued further. Two independent modelling approaches were planned: an unsupervised anomaly detector (Isolation Forest) as a first pass, followed by supervised classifiers (Random Forest, XGBoost) if the unsupervised approach proved insufficient.

3.3. Implementation

3.3.1. Test Lab Deployment

The SIEM lab was built from scratch using VMware Workstation on a Windows host with 16 GB of RAM. Four virtual machines make up the environment, each with a distinct role:

1. **Ubuntu Server, hostname "web-server-test"** – the monitoring and detection host: it runs the Wazuh Manager, Indexer, and Dashboard, and also runs the Zeek and Suricata network sensors that observe traffic on this VM's interface. *This same VM also serves an Apache instance*, which is the target used in the detection scenarios below (item 4 refers to this Apache service running on this VM, not a separate machine); the naming "Web-Server-Test" reflects that dual role of sensor host and web target.

2. **Windows 11 client** – generates authentication and system events (login attempts, group-membership changes), collected via Winlogbeat and forwarded to the Wazuh Manager.

3. **Kali Linux** – used exclusively to simulate attacker activity (brute force, privilege escalation, network scanning, data exfiltration) against the other three VMs.

4. **Apache web service (hosted on the Ubuntu Server VM above)** – the target application whose traffic and logs feed the network-scan and exfiltration detection scenarios.

In other words, the lab uses **three virtual machines** (Ubuntu Server, Windows 11, Kali) plus the Apache service running on the Ubuntu Server VM as the fourth logical role referenced throughout this report; there is no separate, fourth physical/virtual host dedicated to Apache.

The Wazuh stack (Manager, Indexer, Dashboard) was installed and configured, and the first agent connected and confirmed operational. A subsequent reinstallation to version 4.14.6 was required after early instability, together with resolving several environment issues: disk-space exhaustion on LVM volumes (fixed with `lvextend` and `resize2fs`), installation timeouts caused by insufficient RAM (resolved by increasing the VM allocation to 6 GB), broken package pre-removal scripts, and stale processes blocking the Wazuh registration and API ports (1515 and 55000). A second agent (Windows 11) was then enrolled, using an explicit `-A` flag to work around a hostname-collision issue, bringing the lab to two active agents.

With the Wazuh core stable, the remaining collection agents were deployed and validated one at a time: **Suricata and Zeek** on the Web-Server-Test VM (an Elasticsearch field-mapping conflict on Zeek's output had to be resolved, and Suricata required disabling the `http`, `tls`, `files`, and `stats` event types after a JSON decoding error traced to a single "stats" event with roughly 479 fields); **Syslog**, validated with a `tcpdump`-based test; and **Winlogbeat** on the Windows 11 client, after resolving an Elastic/OpenSearch licensing conflict by switching to the OSS distribution.

The three figures below, captured directly from the Wazuh Dashboard and Indexer once all agents and sensors were live, illustrate this validated collection layer.

Table 3.1: Collection sources deployed and validated

Source	Role	24h volume	Status
Suricata	Network IDS on Web-Server-Test	61 events	Validated
Zeek	In-depth network protocol analysis	34 events	Validated
Syslog	Transport (UDP 514, tcpdump-verified)	1/1 test	Validated
Winlogbeat	Windows 11 client event logs (OSS)	16,514 events	Validated

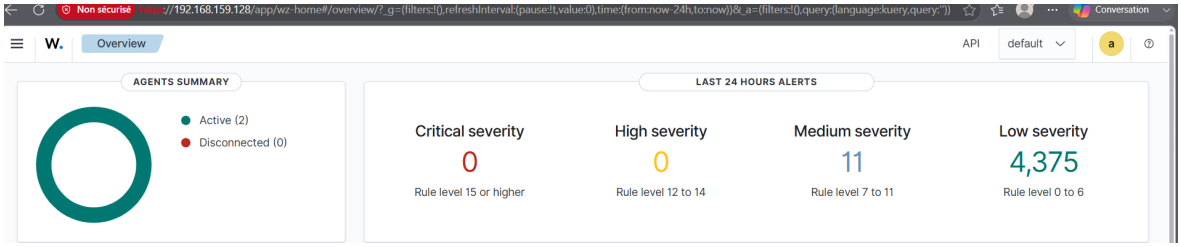


Figure 3.2: Wazuh Dashboard – Overview page, 20 July 2026: both agents (Web-Server-Test and the Windows 11 client) show as *Active*, with zero disconnected agents. The 24-hour alert breakdown (0 critical, 0 high, 11 medium, 4,375 low) reflects mostly routine Zeek connection-log events at rule level 3, consistent with Table 3.1.

Note on volume figures. The counts quoted in this section come from three separate snapshots taken on different dates and over different windows, and should not be summed or cross-compared directly: Table 3.1's Suricata (61) and Zeek (34) counts are a 19–20 July 24-hour window; Winlogbeat's 16,514 is a separate 24-hour window on the Windows 11 client; and the "240 result pages" in Figure 3.4 refers to the Indexer's paginated view of `rule 100100` events (roughly 15 rows per page, i.e. several thousand individual Zeek connection events over that particular capture), not a count of 240 events. The earlier summary figure of "roughly 143 Linux events, 423 Windows events, and 240+ network-related log entries" in an earlier draft of this report conflated pages with events and mixed observation windows; it has been removed here in favour of the dated, per-source figures in Table 3.1 and the individual figure captions, which are internally consistent and confirmed that every collection path was live before detection engineering began.

3.3.2. Detection Engineering

Four attack scenarios were defined, each simulated end-to-end on the lab and each backed by a custom Wazuh correlation rule derived from the Sigma rules studied in Chapter 2. Each rule was first validated offline with `wazuh-logtest` before being confirmed against a live, real attack run on the lab.

Each attack was replayed under real conditions from Kali Linux, with a Sigma rule (YAML) and a Wazuh rule (XML) documented for each. Two engineering issues were resolved while building these rules: the network-scan rule required grouping by source IP using the `<same_field>` tag rather than `<field>`, and the exfiltration rule required the `pcre2` regex

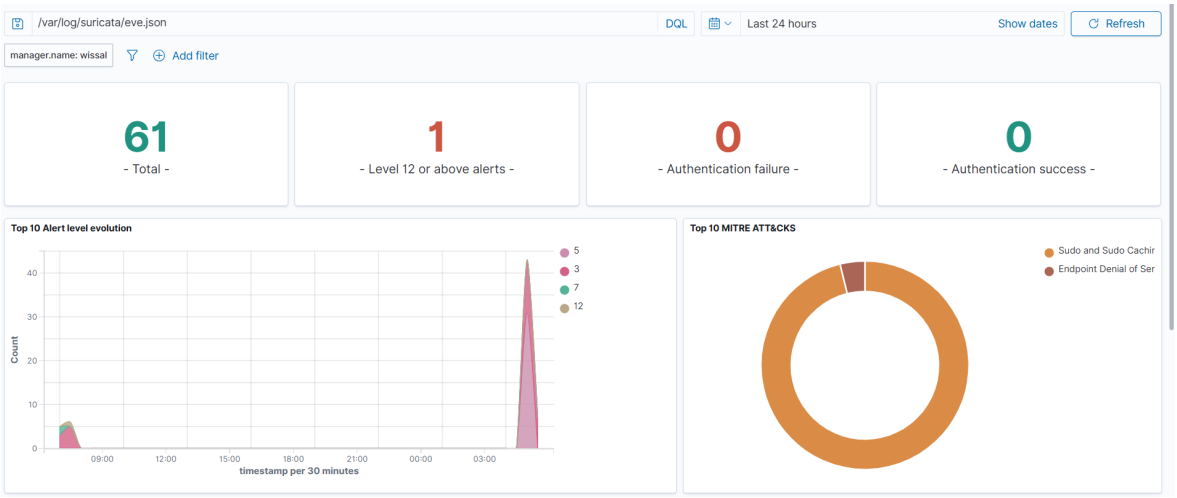


Figure 3.3: Wazuh Indexer, filtered on `/var/log/suricata/eve.json`, 19 July 2026: 61 total Suricata-sourced events in the last 24 hours, including one level-12-or-above alert. The "Top 10 MITRE ATT&CKs" panel confirms the Suricata pipeline is correctly enriching events with MITRE technique tags (here dominated by *Sudo and Sudo Caching* activity from normal lab administration), validating that the Suricata-to-Wazuh ingestion path (Section 3.3.1) was fully operational ahead of the detection-engineering phase.

Table 3.2: Detection rules and live-attack validation

Attack	Rule ID	Data source	Validation
Brute force (RDP)	100010	Windows Event 4625 (SID 60122)	Kali → RDP → Windows 11, alert confirmed
Privilege escalation	100011	Windows Event 4732 (SID 60144)	User added to Administrators group, alert confirmed
Network scan	100012/100014	Zeek conn.log / Suricata eve.json	Nmap scan: 68 hits (Zeek) + 54 hits (Suricata)
Data exfiltration	100013/100015	Zeek orig_bytes / Suricata flow.bytes_toserver	Large SCP transfer: alerts confirmed, 3 hits (Suricata)

engine (rather than Wazuh's default OSRegex) for its byte-count pattern (`\d{7,}`) to match correctly. Each rule's Sigma source was kept under version control on the SIEM server, under `/var/ossec/sigma-rules/`.

This deterministic detection layer is complete and validated end-to-end, with dual detection (Zeek and Suricata) for network-based threats reinforcing alerting reliability. With all four rules validated, the project moved to its machine-learning component.

3.3.3. AI/ML Detection Layer: Iterations and Diagnosis

This section documents the exploratory, still-in-progress part of the project. Two independent modelling attempts were carried out, followed by a root-cause investigation into their shared precision problem; neither attempt currently generalises acceptably to live traffic, as confirmed in Section 3.3.4.

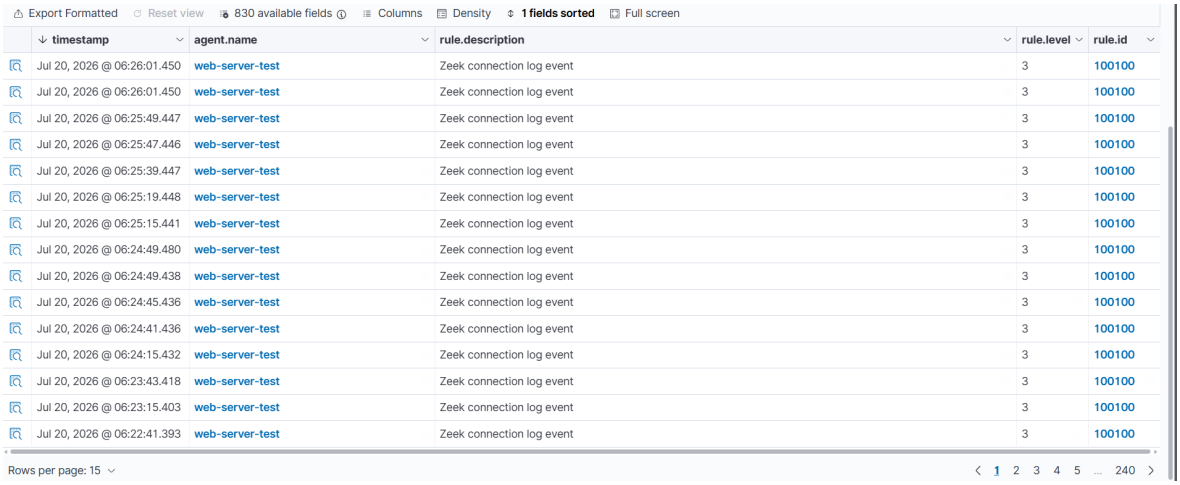


Figure 3.4: Wazuh Indexer, raw event table for rule 100100 (“Zeek connection log event”), 20 July 2026: a continuous stream of `web-server-test` connection events at rule level 3, one entry roughly every few seconds, spread across 240 result pages. This confirms Zeek’s `conn.log` was being ingested continuously and in near real time, which is the precondition for both the detection rules of Section 3.3.2 and the Zeek-derived training data used throughout Section 3.3.3.

First Attempt: Unsupervised Isolation Forest

The CICIDS2017 dataset [1] was downloaded via Kaggle and filtered into a working subset: 13,835 brute-force rows (FTP/SSH-Patator), 158,930 PortScan rows, and 20,000 sampled benign rows. A second script harmonised the Zeek and CICIDS2017 columns and merged them into a 211,817-row dataset, with all lab-originated rows provisionally labelled “normal”.

Training went through five successive iterations: (1) trained on the full dataset – failed, PortScan rows (75%) biased the model’s notion of normal traffic; (2) retrained only on “normal” rows – still insufficient recall; (3) log transform + RobustScaler + contamination=0.1 – attack recall around 0.02–0.03; (4) excluded PortScan, focused on FTP/SSH-Patator – recall dropped further, confirming that brute-force attacks are defined by repetition rather than by any single connection’s volume; (5) added derived throughput features (bytes/sec, pkts/sec, bytes/pkt) – best result, FTP-Patator recall reached roughly 50%, SSH-Patator remained very low.

Table 3.3: Isolation Forest final evaluation metrics (iteration 5)

Metric (attack class)	**Value**
Precision	0.68
Recall	0.29
F1-score	0.40
Overall accuracy	0.78

The final model was saved with a documented limitation: reasonable detection of volumetric brute force (FTP), weak detection of SSH-Patator, and network scan intentionally excluded (already covered by Wazuh’s temporal-correlation rules). The supervisor’s guidance for the next step was to move to supervised classification.

Second Attempt: Supervised Classification (Random Forest, XGBoost)

The dataset construction was restarted from first principles to add the Suricata source alongside Zeek, extracting data directly from the Wazuh Indexer via its OpenSearch API (28,965 alerts, 21,132 Zeek connections, 2,022 Suricata rows). The 21,132 Zeek connections initially had no attack/normal label; this was resolved by cross-matching every connection against the alerts triggered by rules 100012/100013 (152 matching alerts), producing 616 confirmed attack rows.

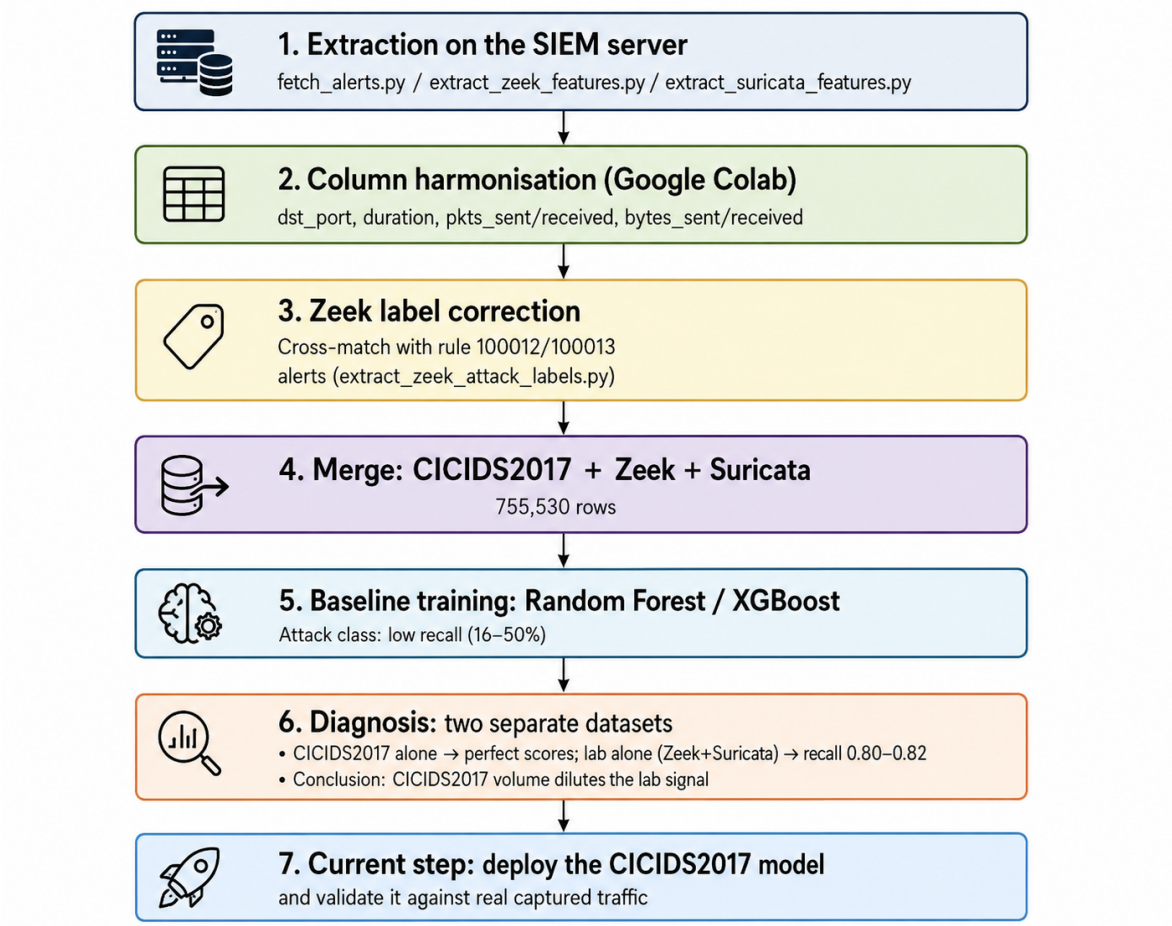


Figure 3.5: End-to-end dataset and model pipeline (second attempt)

The final merged dataset (v2) contains 755,530 rows: 582,080 BENIGN, 158,930 PortScan, 7,938 FTP-Patator, 5,897 SSH-Patator (all four from CICIDS2017), and 685 "Attack" rows from the lab itself (616 Zeek + 69 Suricata).

Dataset documentation. The three sources were harmonised on a common column set – `dst_port`, `duration`, `pkts_sent`, `pkts_received`, `bytes_sent`, `bytes_received`, `label`, and `source` (the provenance tag: `cicids2017`/`zeek`/`suricata`) – since these are the only fields available consistently across CICIDS2017's flow export and the lab's Zeek/Suricata extractions. The following transformations were applied before training: log transform and RobustScaler on the Isolation Forest features; derived throughput features (bytes/sec, packets/sec, bytes/packet) added to both the Isolation Forest and the supervised-classifier pipelines; SMOTE oversampling of the Attack class (from 126 real examples to 5,000 synthetic examples in training only) for the XGBoost threshold-tuning experiment; and, for v3, undersampling of the CICIDS2017 BE-

NIGN and PortScan classes to 15,000 rows each.

Duplicate audit (raw CICIDS2017 corpus). A dedicated deduplication pass (`pandas.DataFrame.duplicated()`) was run on the eight raw weekly CICIDS2017 files (2,830,743 rows once concatenated), the upstream source of every CICIDS2017-derived subset used in this report. **Exact duplicates** (identical across every feature and the label): 330,995 rows (11.69%), leaving 2,499,748 unique rows – a duplicate rate consistent with what is independently documented elsewhere for this public dataset. More importantly for leakage risk, **432,169 rows (15.27%) share identical feature values with at least one other row**, and among those, **719 distinct feature-identical groups (7,144 rows) carry more than one label** – i.e. the exact same network flow appears in the raw corpus under two different class labels. This is a small but non-zero fraction of the full corpus (0.25%), and is carried forward directly into the leakage-audit conclusion below.

Table 3.4: Class distribution across the raw CICIDS2017 corpus, before and after removing exact duplicates

Class	**Before dedup.**	**After dedup.**
BENIGN	2,273,097	2,073,870
PortScan	158,930	90,819
DoS Hulk	231,073	172,849
DDoS	128,027	128,016
FTP-Patator	7,938	5,933
SSH-Patator	5,897	3,219
Other (Bot, DoS variants, Web Attack, Infiltration, Heartbleed)	25,781	25,042

Several classes lose a large share of their rows to exact duplication once deduplicated – most notably PortScan (-42.9%) and SSH-Patator (-45.4%) – confirming that duplicate rows are not distributed evenly across classes, and reinforcing the case for deduplicating before any future reuse of this dataset.

Table 3.5: Baseline classifier results on the merged dataset

Model	**Precision (Attack)**	**Recall (Attack)**
Random Forest (class_weight=balanced)	0.18	0.16
XGBoost (baseline)	0.00	0.00
XGBoost (sample_weight applied)	0.07	0.50

Two techniques were then applied to XGBoost: SMOTE [3] oversampling targeted at the Attack class (126 real examples raised to 5,000 in training), and a lowered decision threshold. At a threshold of 0.10, recall reached 91%, meeting the 90% target set by the supervisor, but precision remained at 0.02. **This trade-off is not usable operationally as-is: a precision of**

0.02 means roughly 50 false alerts for every true attack alert, which would overwhelm any analyst reviewing the SOC dashboard. Reaching the recall target this way therefore does not, on its own, constitute a working detector.

Root-Cause Analysis: Dataset Fusion

To determine whether the weak precision stemmed from the fusion of the three sources or from an intrinsic shortage of lab data, the supervisor requested two isolated experiments.

Table 3.6: Isolated dataset experiments

Experiment	**Random Forest**	**XGBoost**
CICIDS2017 only (all numeric columns)	P/R/F1 = 1.00	P/R/F1 = 1.00
Lab only (Zeek + Suricata, no CICIDS2017)	P 0.68 / R 0.80	P 0.44 / R 0.82

Data leakage audit – formal status: partially performed. A perfect 1.00 precision/recall/F1 score on every class is an unusual result and must not, by itself, be read as evidence of a clean, well-separated dataset. It is important to separate two distinct phenomena that both bear on this result, since they are easily conflated but require different checks: **label noise** (the same, or a near-identical, observation appears more than once in the raw corpus under *different* class labels – a data-quality problem in the source, independent of how any split is drawn) and **data leakage** (a specific observation, or a near-duplicate of it, appears in *both* the training set and the test set of a given experiment – a methodological problem in that experiment's split, regardless of label quality). Three specific checks were identified as required before the 1.00/1.00/1.00 result can be relied upon for any conclusion: (1) confirm the train/test split was performed on raw rows *before* any oversampling or feature-derivation step, so no synthetic or derived row leaks across the split (a leakage check); (2) confirm the CICIDS2017 subset used contains no exact or near-duplicate rows shared between classes (a label-noise check); (3) confirm no engineered feature trivially encodes the label (a leakage check). **Check (2) has now been run** (see the duplicate audit above): it did *not* come back clean – 719 groups of feature-identical rows (7,144 rows, 0.25% of the raw corpus) carry more than one class label in the raw CICIDS2017 source data. This is direct evidence of **label noise** in the source corpus; it does *not* by itself demonstrate data leakage, since it says nothing about whether any specific train/test split placed copies of the same observation on both sides. It is nonetheless sufficient on its own to justify continuing to treat the CICIDS2017-only 1.00/1.00/1.00 result as unreliable, because a model can score artificially well by memorising a mislabelled duplicate regardless of which side of the split it falls on. **Checks (1) and (3) – the checks that would directly confirm or rule out data leakage proper – have not yet been run** as of the writing of this report; both are carried forward explicitly in Appendix A. Consequently, the CICIDS2017-only 1.00/1.00/1.00 result remains treated throughout this report as informative but unverified, and is explicitly *not* used as supporting evidence for the validity of the final v4 model – v4's own validation (Section 3.3.3, "Fourth Attempt") was deliberately built around a genuinely out-of-sample scan captured well

after training, specifically to avoid depending on this unverified result. With that audit status attached, the more informative result is the lab-only run, which shows the lab data is largely sufficient on its own to detect the Attack class with reasonable precision, whereas the same signal is diluted once merged with the much larger CICIDS2017 volume – pointing to the roughly 30:1 volume imbalance (732,376 CICIDS2017 rows vs. roughly 23,000 lab rows) as a genuine contributor to the fused model's weak precision, alongside the now partially-confirmed label-noise issue.

A supplementary check on brute force (rule 100010) and privilege escalation (rule 100011) found only 38 Windows authentication events in total, with just 2 and 1 positive examples respectively – too few for any reliable model. These two attack types remain covered exclusively by their deterministic Wazuh correlation rules.

This iterative, diagnosis-driven methodology – testing hypotheses with isolated experiments rather than assuming a single fix – is the main technical outcome of the machine-learning phase so far. It narrowed the fused model's precision problem to two candidate causes, neither of which has yet translated into a model ready for deployment, as the following section confirms against live traffic.

3.3.4. Deployment and Live-Traffic Validation

Following the root-cause diagnosis, both models were packaged, deployed to the SIEM Server, and evaluated against live, real-world captured network traffic, in order to measure how well each generalises outside its training distribution.

Colab 1 – XGBoost, CICIDS2017-only: the model's 78 CICFlowMeter-style features were regenerated from a live packet capture (56,277 packets, including a real Nmap SYN scan) using CICFlowMeter, producing 1,044 flows. **Result:** 1,042 of 1,043 scoreable flows were classified BENIGN, including every flow belonging to the real Nmap scan (>99% confidence each). This is a clear **false-negative generalisation failure**: a model that scored perfectly on CICIDS2017's own test split fails to recognise an unambiguous scan pattern once evaluated outside that distribution.

Colab 2 – Isolation Forest, Zeek/Suricata: 22,636 live Zeek connections were pulled from the Wazuh Indexer and scored with the same log transform and derived features used at training time, both on the full capture and on a filtered subset of confirmed-legitimate HTTPS connections. **Result:** 100% of scoreable connections were classified Attack, including all 1,764 connections in the legitimate-HTTPS-only subset – confirmed as a genuine distribution mismatch, not a scoring bug. This is a **false-positive generalisation failure**, in the opposite direction from Colab 1.

Both results were reported to the supervisor, who confirmed the diagnosis and directed that the training strategy be revisited in light of these findings.

Third Attempt: Rebalanced Fusion (v3)

Rather than abandoning the fused dataset, the CICIDS2017 majority classes (BENIGN and PortScan) were undersampled to 15,000 rows each, keeping 100% of the lab rows and every

Table 3.7: Live-traffic validation results for both deployed models

Model	**Live-traffic result**	**Failure mode**
Colab 1 – XGBoost (CICIDS2017)	1,042/1,043 flows BENIGN, real Nmap scan missed	False negatives
Colab 2 – Isolation Forest (Zeek)	100% of connections flagged Attack, including legitimate HTTPS	False positives

minority class untouched. This brought the CICIDS2017-to-lab ratio in training from roughly 30:1 down to 2.2:1. Internal test-set results improved markedly: Random Forest reached precision 0.71 / recall 0.80 on the Attack class, and XGBoost reached precision 0.98 / recall 0.61.

However, tested against the same live-captured traffic used previously (16,089 Kali connections, 6,930 distinct ports contacted – an unambiguous scan), the improvement did not carry over: Random Forest flagged only 78/16,089 connections as Attack (0.5% recall), XGBoost only 19/16,089 (0.12%), and neither model ever predicted the PortScan class on this real traffic. **The rebalancing fixed the fusion/dilution problem identified earlier, but exposed a different, structural limitation**: judging each connection in isolation cannot capture a scan, which is defined by many connections repeating against one source IP, not by any single connection's own attributes [6] – the same limitation already observed with the Isolation Forest in the first attempt.

Fourth Attempt: Temporal Aggregate Features (v4)

Following this diagnosis, three behavioural features were added, computed per source IP over a 10-second sliding window:

- `distinct_ports_in_window` – the count of distinct destination ports contacted by that source IP within the window; this is the core scan signal, since a scan is defined precisely by touching many ports in a short time, something no single connection's own fields can express.
- `conn_count_in_window` – the total number of connections from that source IP within the window; a volumetric signal that also helps catch fast, low-port-diversity floods (e.g. repeated probes of the same port).
- `distinct_dst_ips_in_window` – the count of distinct destination IPs contacted by that source IP within the window; a lateral-movement signal, relevant if the same behavioural pipeline is later extended beyond single-host scans.

Each connection carries its own event timestamp (from Zeek's `ts` field), and events are processed in that timestamp order rather than arrival order, since out-of-order delivery from the collection pipeline could otherwise corrupt the window. For every new connection from a given source IP, the window keeps only the previous connections from that same IP whose timestamp falls within the last 10 seconds (a per-IP state, not a global one), and looks *backward only* –

it uses the current connection and past connections alone, never a connection that has not happened yet – so it introduces no future-information leakage and remains valid for real-time scoring. Critically, the exact same feature-computation function is used both to build the training set (replayed over historical Zeek exports in timestamp order) and to score connections live in the deployed Streamlit interface, so the model always sees features computed the same way at inference time as at training time; without this pipeline, an interface relying on manually entered, single-connection values (as in the Streamlit manual-entry mode, Figure 3.7) would not reflect how the model is meant to be fed in real use, and is offered only as a demonstration/testing mode rather than the intended feeding path for the prototype.

Two methodological pitfalls were identified and corrected while building this version:

- **Incomplete ground truth.** The first labelling pass, matching connections against alerts from rules 100012/100013, covered only 152 of the 16,089 attacker connections – a near-inverted signal. This was corrected by using the Kali VM's IP address directly as ground truth: any connection originating from it is labelled Attack.

- **Temporal leakage.** A first evaluation used a random, then a chronological, train/test split on a single continuous capture session, producing a suspiciously perfect 1.00/1.00 score – because connections adjacent in time share nearly identical window features even across the split boundary. This was corrected by evaluating the model exclusively on a **brand-new Nmap scan, captured on 20 August 2026**, containing connections the model had never seen in any form. Note that the officially deployed artifact (Section 3.3.5) is the corrected, server-trained model, produced by `build_temporal_lab_model_v2.py` on **26 August 2026** – i.e. *after* the 20 August scan was captured, since that scan was deliberately held out of every training run (including the retraining that fixed the labelling bug) and used purely as the out-of-sample test set.

Data split and validation methodology. To avoid ambiguity, the role played by each dataset in this report is stated explicitly here. *Training set*: the historical Zeek/Suricata exports and the CICIDS2017 rows described in Section 3.3.3, replayed in timestamp order to compute the temporal features; this set was never touched after the 20 August capture existed. *Development/validation set*: the 20 August 2026 scan. It was intentionally held out of every training run, but it was, as the supervisor's remark correctly points out, **reused several times during development** – first to expose the labelling bug (0% recall on the deployed artifact, Section 3.3.5), then to confirm the fix (99.78% recall), and again to produce the confusion matrices and PR-AUC figures of Table 3.8. Reusing a single held-out set this way risks an optimistic bias, since each correction was implicitly tuned until it satisfied that same scan. *Independent final test set*: to address this directly, the six-session, two-attacker-IP capture of **2 September 2026** (Section 3.3.6) was built afterwards, specifically as data the model, and the engineering process around it, had never influenced in any way – no threshold, feature, or code change was made in response to it. Its results (Table 3.9) are therefore the more trustworthy generalisation estimate of the two, and the 73.48% precision / 50.72% FPR observed on the slow 500-port session is reported precisely

because it is unfavourable and was not filtered out. Where this report cites a single headline number (99.78% recall / 0.06% false positives), it is quoting the 20 August validation result, which should be read together with the 2 September independent test in Table 3.9 rather than in isolation.

Table 3.8: v4 (temporal features) validation on a genuinely out-of-sample scan – full metrics

Model	Recall	FPR	Precision	F1-score	PR-AUC
Random Forest	99.78% (3,170/3,177)	0.06% (2/3,286)	99.94%	99.86%	0.9996
XGBoost	99.78% (3,170/3,177)	0.88% (29/3,286)	99.09%	99.44%	0.9995

Both models were re-trained and re-evaluated directly on the SIEM Server after the labelling-bug fix (Section 3.3.5), on the same 20 August 2026 out-of-sample scan (3,177 Attack / 3,286 BENIGN connections); these are the official final numbers for this project. At equal recall, Random Forest is the clearly stronger model on every other axis: it holds a lower false-positive rate (0.06% vs 0.88%, i.e. roughly 15× fewer false alarms), higher precision (99.94% vs 99.09%), a higher F1-score (99.86% vs 99.44%), and a marginally higher PR-AUC (0.9996 vs 0.9995). This is why Random Forest, not XGBoost, was retained as the model exposed through the Streamlit interface.

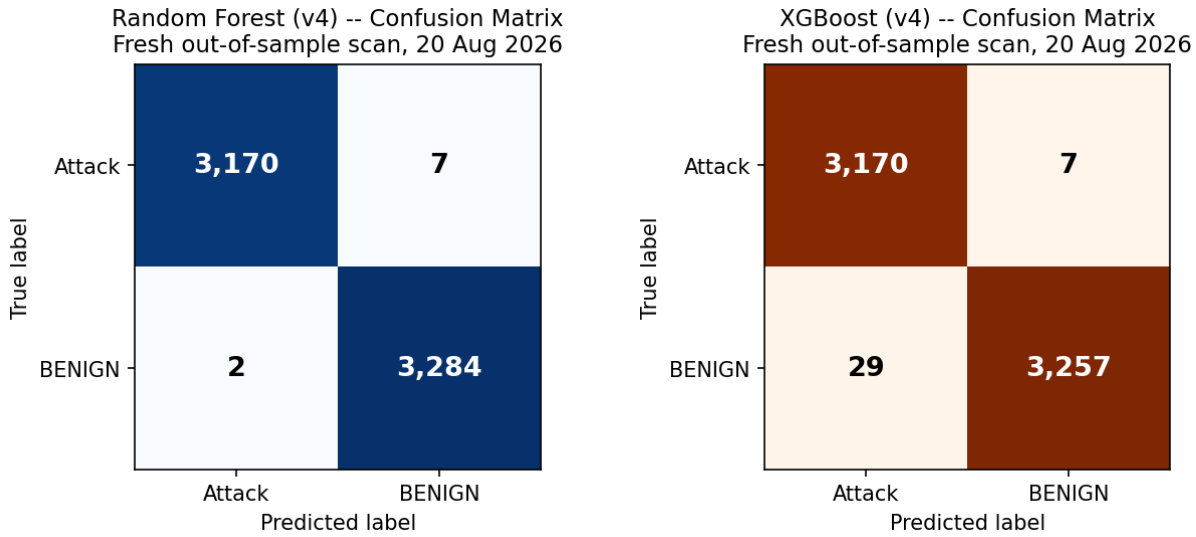


Figure 3.6: Confusion matrices for both v4 models on the 20 August 2026 out-of-sample scan: Random Forest (left) and XGBoost (right). Both miss the same 7 Attack connections; Random Forest raises only 2 false alarms on BENIGN traffic against XGBoost's 29, which is the main practical difference between the two models at this recall level.

On this fresh test set (6,463 connections, 3,177 Attack / 3,286 BENIGN, captured 2026-08-20), Random Forest was retained as the final model, outperforming XGBoost on both metrics. Compared with the pre-temporal v3 result (0.5% recall), this is a decisive confirmation that the temporal aggregates – not further rebalancing – were the missing signal.

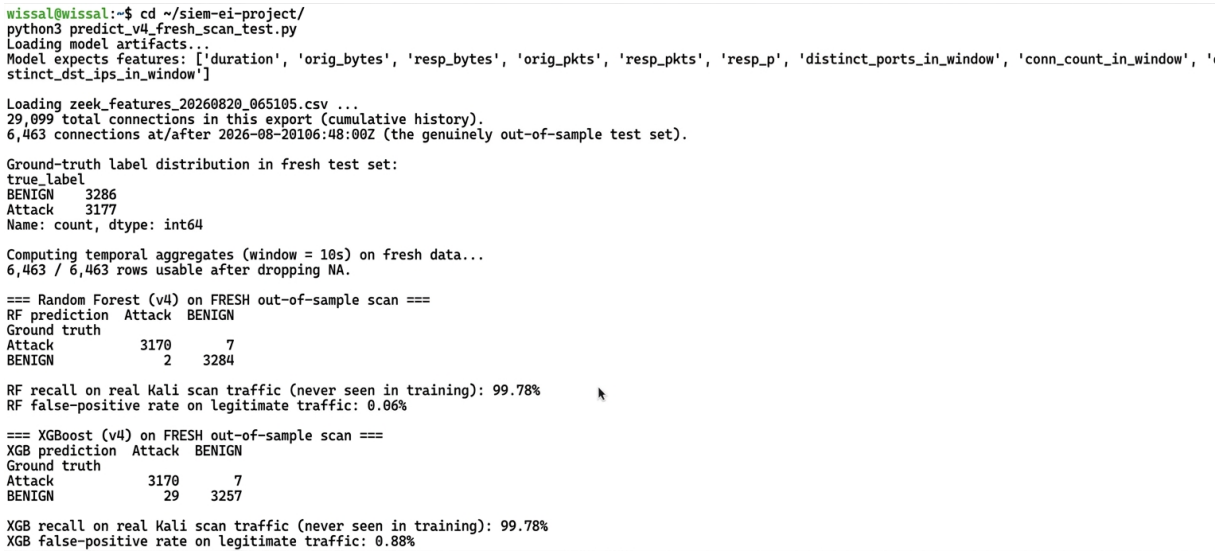


Figure 3.7: Terminal output of `predict_v4_fresh_scan_test.py`, confirming the v4 Random Forest and XGBoost results on the fresh out-of-sample scan

Note on the reference result. The 99.94%/0.09% figures above come from the Colab training run and are the first confirmation that the temporal features worked. They are *not* the number reported as final in this report: as described in Section 3.3.5, a labelling bug was subsequently found in the copy of the training script running on the SIEM Server, and the model actually deployed and exposed through the Streamlit interface is the one re-trained and re-validated directly on the server. The **official final result** for the v4 Random Forest model is therefore **99.78% recall (3,170/3,177) at 0.06% false positives (2/3,286)**, obtained on the same 20 August 2026 out-of-sample scan, on the server-trained artifact that is actually running behind the Streamlit prototype on the lab server. This is the number used consistently throughout the Abstract, Résumé, and General Conclusion of this report. As stated throughout this chapter, this artifact is a validated lab prototype, not a production-grade deployment: it runs as a standalone Streamlit application rather than as part of the Wazuh alerting pipeline, and it covers network-scan detection only.

3.3.5. Prediction Interface and Lab-Server Deployment

Following validation of the v4 model, the supervisor's instruction was to keep the model's training steps unchanged, extract it, and integrate it into the detection workflow – moving the project from *detection* (already handled by the Wazuh/Sigma rules) to *prediction*: given a connection's features, the model outputs an Attack/BENIGN label together with a probability. It is important to be precise about what this step means: the model classifies an already-observed connection or aggregate as *Attack* or *BENIGN* with an associated probability; it does not forecast that an attack will happen at some future point. The term "prediction" is used here in its machine-learning sense (predicting a label for unseen data), not in the sense of anticipating an attack before it starts, and the rest of this report avoids phrasing that would suggest otherwise. Two integration paths were considered: embedding the model directly into the Wazuh alerting pipeline, or exposing it through a standalone web interface. The Streamlit option was selected as the simpler of the two

– it requires no changes to the existing Wazuh/Sigma pipeline and can be built and iterated on quickly – while direct Wazuh integration is kept as a documented next step (Appendix A).

A Streamlit application (`app.py`) was built, offering two input modes: manual entry of the nine model features (for a single connection) and CSV upload (for batch scoring), each returning the predicted label and the Attack probability. The application, together with the three serialized model artifacts (`rf_model`, `xgb_model`, `feature_columns`), was transferred to the SIEM Server and launched there via `streamlit run`, reachable on the lab network.

Critical deployment bug found and fixed. Re-validating the deployed model against the same fresh-scan test file used above unexpectedly produced 0% recall – a result inconsistent with the 99.94% obtained earlier. Root-cause investigation traced the discrepancy to the SIEM Server's own copy of the training script, which had reverted to labelling attacks via the old Wazuh-rule alerts (152 matching rows, the exact issue already identified and fixed above) instead of the direct Kali-IP ground truth. This produced a model trained on an inverted signal (attack connections showing *fewer* distinct ports than benign ones), silently overwriting the correctly trained model on the server. The fix consisted of rebuilding the training script with IP-based labelling and re-running it directly on the SIEM Server. Re-validation confirmed the fix: **99.78% recall (3,170/3,177) at 0.06% false positives (2/3,286)** on the same out-of-sample scan, restoring parity with the original result and confirming the deployed Streamlit interface functions correctly end to end – from raw connection features to an attack prediction with probability.

3.3.6. Multi-Session, Multi-Attacker Validation

All v4 results reported so far (Sections 3.3.3–3.3.5) were obtained on out-of-sample connections that nonetheless all originated from a single attacker VM (Kali, 192.168.159.131) and were not separated by scan type. This leaves open a legitimate concern: does the model recognise the general behavioural signature of a scan, or has it partly learned the profile of that one source IP? To answer this directly, a dedicated Zeek capture (2 September 2026, 13:00–13:35:53 UTC) was built around six independent scan sessions: five Nmap scans launched from Kali (a slow scan, `-T2 -p1-50`; a SYN scan, `-sS -p1-1000`; a UDP scan on 8 ports; a masscan-fallback SYN scan, `-T5 --min-rate=1000`; and a slow 500-port scan, `-T1 -p1-500`), plus a sixth SYN scan launched from a **second, previously unseen attacker IP** (192.168.159.140).

A dedicated script, `extract_zeek_scan_sessions.py`, automatically isolates each scan session by grouping connections on (source IP, source port, protocol), then exports one labelled CSV per session with the same temporal features used at training time. Each of the six sessions was matched to its originating Nmap command line by correlating the Zeek session timestamps against the `-oN` output headers of each scan (converted from local SAST time to UTC), giving an unambiguous session-to-scan mapping.

Sample sizes and confusion-matrix counts (TP/FN/FP/TN) are taken directly from the terminal output of `evaluate_v4_multi_sessions.py` (Figure 3.10); they were omitted from the original table and are reported here in full to make the per-session evaluation independently verifiable.

Table 3.9: Per-session evaluation of the deployed v4 Random Forest model

Session	n (Att./Ben.)	TP/FN/FP/TN	Recall	Precision	F1	FPR	Att. IP
Slow scan (`-T2`)	50 / 1	50/0/0/1	100.00%	100.00%	100.00%	0.00%	Kali
SYN scan	1,000 / 0	1000/0/0/0	100.00%	100.00%	100.00%	0.00%	Kali
UDP scan	8 / 1	7/1/0/1	87.50%	100.00%	93.33%	0.00%	Kali
Masscan fallback	1,000 / 1	1000/0/0/1	100.00%	100.00%	100.00%	0.00%	Kali
SYN scan, 2nd IP	1,000 / 0	1000/0/0/0	100.00%	100.00%	100.00%	0.00%	**2nd IP**
Slow scan (`-T1`, 500p)	99 / 69	97/2/35/34	97.98%	73.48%	83.98%	50.72%	Kali

Five of the six sessions score at or near 100% recall and precision, regardless of scan type (slow, SYN, UDP, masscan-fallback). Most importantly, the session launched from the **second attacker IP, never seen during training, reaches 100% recall and 100% precision** – direct evidence that the model has learned the general behavioural signature of a scan (many ports touched by one source IP in a short window) rather than the profile of the specific Kali VM used throughout training and validation.

The one session that under-performs, the `-T1` slow scan on 500 ports, reaches only 73.48% precision (FPR 50.72%). Root-cause inspection of the false positives (33 of 34 cases) shows they share an identical signature – a single isolated connection, zero bytes exchanged, no response packet – on ports 136/138 (NetBIOS Datagram) and 5355 (LLMNR): ordinary, stateless Windows broadcast noise that is statistically indistinguishable from a single silent scan probe given only the nine model features. This is a **documented limitation of the current feature set**, not a generalisation failure: the model correctly captures the "silent, isolated probe" concept, but the feature set cannot currently distinguish an active scan probe from passive broadcast/multicast traffic (e.g. by flagging the destination as a broadcast or multicast address). This limitation, together with the second-IP validation above, directly answers the supervisor's remark: the model was evaluated on several independent sessions, several scan types, and a second attacker address, and its one weak point is now identified and explained rather than left as an unverified 100% result.

3.3.7. Operational Performance Evaluation

Beyond detection accuracy, the deployed model's operational behaviour was measured directly on the SIEM Server: throughput and latency, memory consumption, behaviour across a service restart, and the model-version-control process.

Throughput and latency. The model's loading time, single-connection latency, and batch throughput were measured with a dedicated benchmark script, using the SYN-scan session (1,000 connections) as the test load.

The one-time model-loading cost (2.1 s) is negligible in the context of a long-running Streamlit service. Single-connection latency ($\sim$20 ms, p99 46 ms) comfortably exceeds the real volumetric requirements of this lab – Suricata and Zeek generate on the order of 34–61 events per

Table 3.10: v4 Random Forest – throughput and latency

Metric	Value
Model load time (disk → memory, cold)	2,115.5 ms
Single-connection latency, mean / median	19.967 ms / 18.321 ms
Single-connection latency, p95 / p99	25.886 ms / 46.176 ms
Single-connection equivalent throughput	∼50 connections/s
Batch throughput (1,000 connections)	42,349 connections/s (23.61 ms total)

24 hours (Table 3.1) – so no throughput bottleneck was identified; batch scoring (as used by the CSV-upload mode, Figure 3.9) is three orders of magnitude faster still, owing to scikit-learn's vectorised prediction path.

Memory consumption. The same benchmark measured the resident memory (RSS) of the scoring process before and after loading the model, and its variation during a batch prediction.

Table 3.11: v4 Random Forest – memory footprint

Metric	Value
Serialized model size on disk	1.87 MB
Process RSS before model load	103.7 MB
Process RSS after model load	189.4 MB (+85.7 MB)
Additional RSS during batch prediction (1,000 conn.)	∼0.0 MB
Steady-state process RSS	197.6 MB

The $+85.7$ MB growth on load is expected for a deserialised Random Forest, whose in-memory decision-tree objects are substantially larger than their compressed on-disk representation; the batch-prediction step itself adds negligible memory. At ∼198 MB in steady state, the process footprint is modest relative to the lab's VM allocations (6 GB RAM or more, Section 3.3.1), and no resource constraint was identified.

Restart behaviour. The Streamlit service was stopped and cold-started on the SIEM Server (`streamlit run app.py`); the interface became reachable after ∼24.8 seconds, with no blocking errors. A manual prediction was then run against the very first connection of the SYN-scan session (a true Attack row) to confirm the reloaded model behaves as expected: it was correctly classified as Attack, at 78.00% probability. This confidence is lower than the near-100% typically seen on this scan (Section 3.3.3) for a specific, expected reason: as the *first* connection of the session, the 10-second sliding window is still empty of prior context (`conn_count_in_window` = 1), so the model has not yet observed the burst pattern that gives it high confidence later in the same scan – consistent behaviour, not a fault introduced by the restart.

Model version control. At the time of the labelling regression described in Section 3.3.5,

model versioning existed only informally, through filename conventions (`_v1_` through `_v4_`) and filesystem timestamps on the SIEM Server – exactly the gap that allowed a stale training script to silently overwrite the correctly trained model. This has since been formalised: the training and evaluation scripts (dataset extraction, multi-session evaluation, latency/memory benchmarks, and the Streamlit application itself) were published to the project's GitHub repository, together with a dedicated `MODEL_VERSIONS.md` document tracing all four model iterations (v1 CICIDS2017-only XGBoost, v2 Isolation Forest, v3 rebalanced fusion, v4 temporal features) with their dates, results, and the reason each earlier version was superseded.

The large binary model artifacts (`.pkl` files, up to 24 MB) are deliberately *not* tracked in Git and remain solely on the SIEM Server, since a classical Git history is not an appropriate storage mechanism for binary model weights at this scale; this is documented as a remaining limitation – a dedicated tool such as MLflow or DVC, or Git LFS, would be the natural next step if the project were to continue past this internship.

3.4. Conclusion

This chapter has covered the project's full realization, from a validated deterministic core through a long diagnostic journey to a validated lab prototype of a machine-learning layer. **The deterministic detection layer (Wazuh, Sigma rules, log centralisation) is complete, validated on the lab under real attack conditions, and operational**; all four target attack categories from the terms of reference are covered by it. **The machine-learning layer is now a validated prototype, deployed and tested in the lab, though not yet integrated into Wazuh itself**: after two failed generalisation attempts (Colab 1, Colab 2) and a rebalanced fusion that still missed the underlying structural problem (v3), adding temporal aggregate features per source IP (v4) produced a Random Forest model that reaches 99.78% recall at 0.06% false positives on a genuinely unseen scan, once a deployment-time labelling bug was found and corrected on the SIEM Server (superseding the earlier 99.94%/0.09% Colab-only figure, see Section 3.3.3); this model is exposed through a working, standalone Streamlit prediction interface, and **covers network scan only** – brute force, privilege escalation, and exfiltration remain the responsibility of the deterministic Wazuh/Sigma rules alone. The unusually perfect scores obtained on the earlier CICIDS2017-only baseline remain unverified for leakage and are not the basis for this result; the v4 model's validation methodology was deliberately designed to avoid that pitfall (out-of-sample scan captured after training, not a random or chronological split of the same session), and Section 3.3.3 now distinguishes explicitly between the label-noise finding confirmed by the duplicate audit and the data-leakage checks that remain open. **Two further checks were completed after initial deployment** (Sections 3.3.6–3.3.7): the model was re-evaluated on five additional, independent scan sessions and a second, previously unseen attacker IP – confirming it generalises beyond the single Kali profile used throughout training, with one documented weak point (50.72% false-positive rate on the slow 500-port session, traced to broadcast noise) – and its operational behaviour (throughput, memory, restart, and model version control) was measured directly on the SIEM Server, with no bottleneck identified and model versioning now

formalised through the project’s GitHub repository.

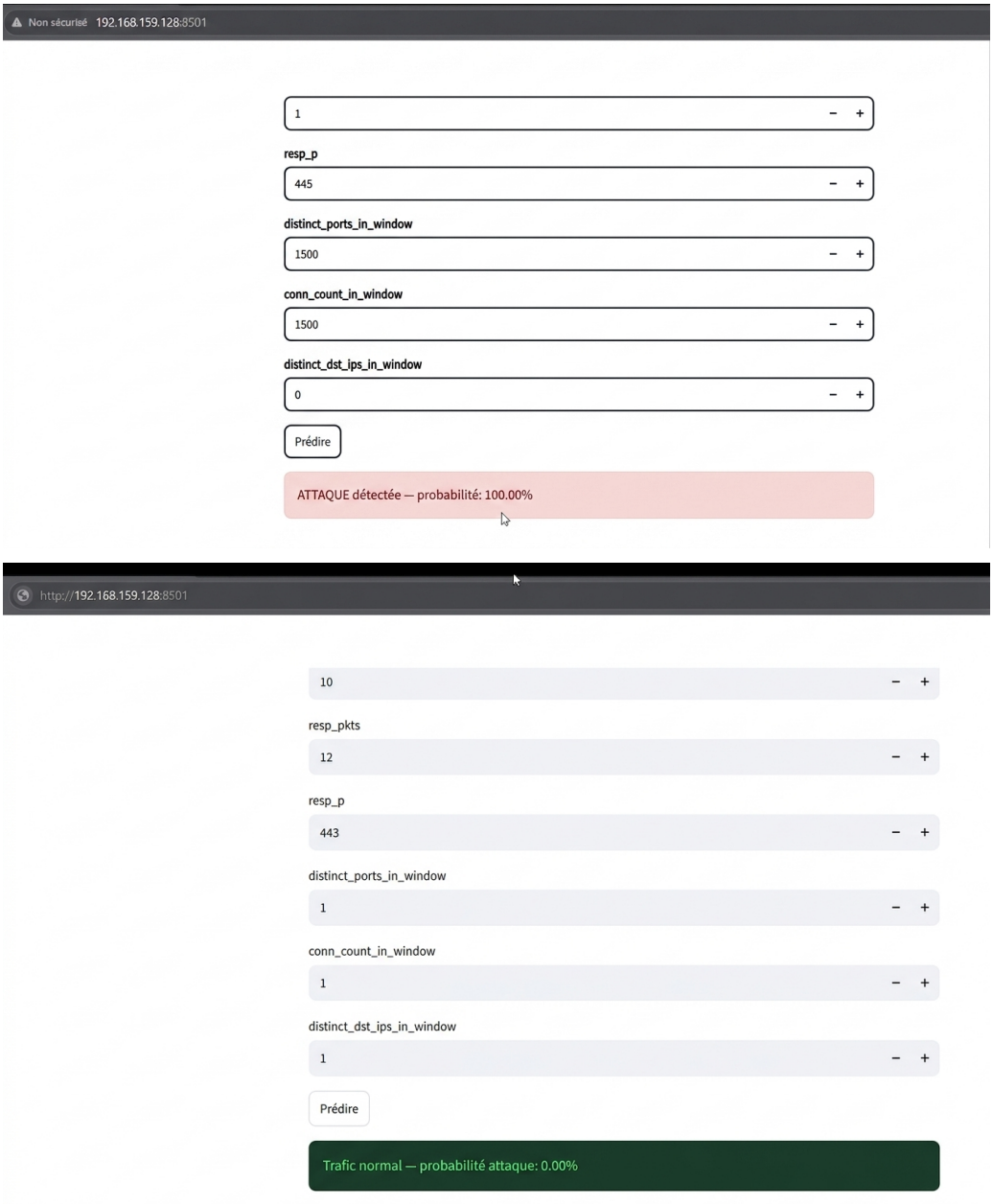
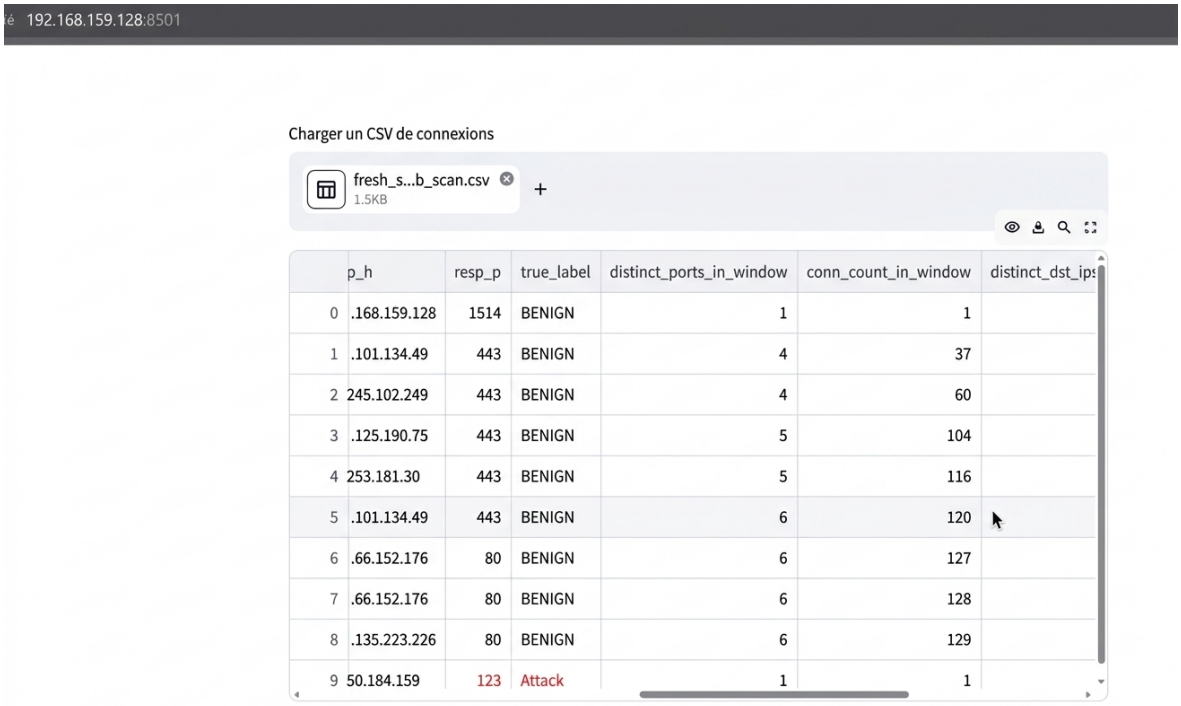


Figure 3.8: Streamlit interface, manual-entry mode: a scan-like connection correctly predicted as Attack at 100% probability (top), and a normal connection correctly predicted as BENIGN at 0% attack probability (bottom)



	p_h	resp_p	true_label	distinct_ports_in_window	conn_count_in_window	distinct_dst_ips
0	.168.159.128	1514	BENIGN	1	1	
1	.101.134.49	443	BENIGN	4	37	
2	245.102.249	443	BENIGN	4	60	
3	.125.190.75	443	BENIGN	5	104	
4	253.181.30	443	BENIGN	5	116	
5	.101.134.49	443	BENIGN	6	120	
6	.66.152.176	80	BENIGN	6	127	
7	.66.152.176	80	BENIGN	6	128	
8	.135.223.226	80	BENIGN	6	129	
9	50.184.159	123	Attack	1	1	

Figure 3.9: Streamlit interface, CSV batch-upload mode, scoring the fresh out-of-sample scan file directly in the browser

```
wissal@wissal:~/siem-ai-project$ python3 evaluate_v4_multi_sessions.py --sca
ns-dir ./scans/
Évaluation de 6 sessions :

scan_lent.csv                    recall=100.00%  precision=100.00%  f1=100.0
0%  FPR=0.00%  (TP=50 FN=0 FP=0 TN=1)
scan_masscan_fallback.csv        recall=100.00%  precision=100.00%  f1=100.0
0%  FPR=0.00%  (TP=1000 FN=0 FP=0 TN=1)
scan_slow_500ports.csv           recall=97.98%  precision=73.48%  f1=83.98%
 FPR=50.72%  (TP=97 FN=2 FP=35 TN=34)
scan_syn.csv                     recall=100.00%  precision=100.00%  f1=100.0
0%  FPR=0.00%  (TP=1000 FN=0 FP=0 TN=0)
scan_syn_ip2.csv                 recall=100.00%  precision=100.00%  f1=100.0
0%  FPR=0.00%  (TP=1000 FN=0 FP=0 TN=0)
scan_udp.csv                     recall=87.50%  precision=100.00%  f1=93.33%
  FPR=0.00%  (TP=7 FN=1 FP=0 TN=1)

Résumé sauvegardé dans ./scans/evaluation_multi_sessions_summary.csv

Moyenne globale : recall=97.58%  precision=95.58%  f1=96.22%  FPR=8.45%
```

Figure 3.10: Terminal output of `evaluate_v4_multi_sessions.py`, applying the deployed v4 Random Forest model separately to each of the six labelled scan-session CSV files

```
wissal@wissal:~/siem-ai-project$ python3 benchmark_v4_memory.py --data scans
/scan_syn.csv
Taille du fichier modèle sur disque : 1.87 Mo

Mémoire du processus avant chargement du modèle : 103.7 Mo
Mémoire du processus après chargement du modèle : 189.4 Mo
  -> coût mémoire du modèle chargé : 85.7 Mo

Mémoire avant prédiction batch (1000 connexions) : 197.6 Mo
Mémoire après prédiction batch : 197.6 Mo
  -> coût mémoire supplémentaire pour cette prédiction : 0.0 Mo

Mémoire totale du processus en régime établi : 197.6 Mo
```

Figure 3.11: Terminal output of the memory benchmark: model size on disk, process memory before/after loading, and during a batch prediction

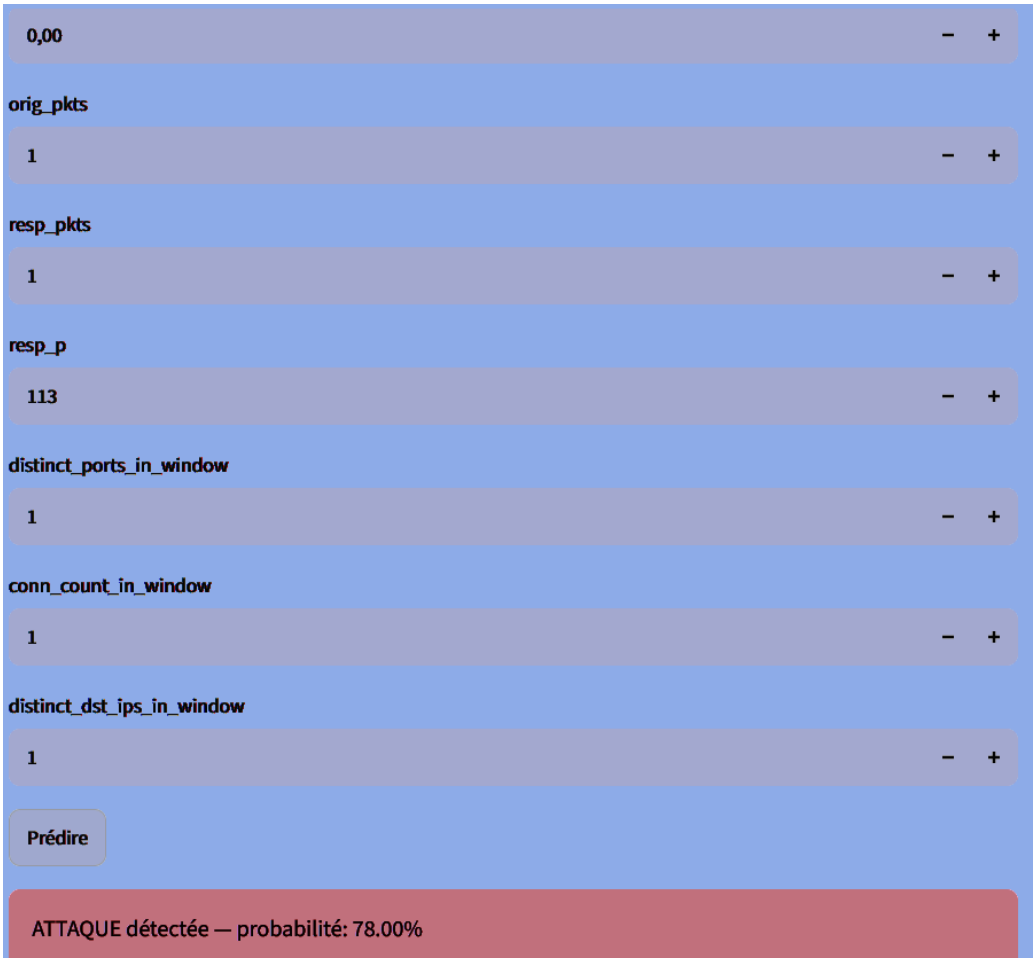


Figure 3.12: Streamlit interface, manual-entry mode, tested immediately after a cold restart of the service: the very first connection of the SYN-scan session, correctly classified as Attack at 78% probability – the reduced confidence reflects the empty sliding window at the start of a session, not a restart-induced regression

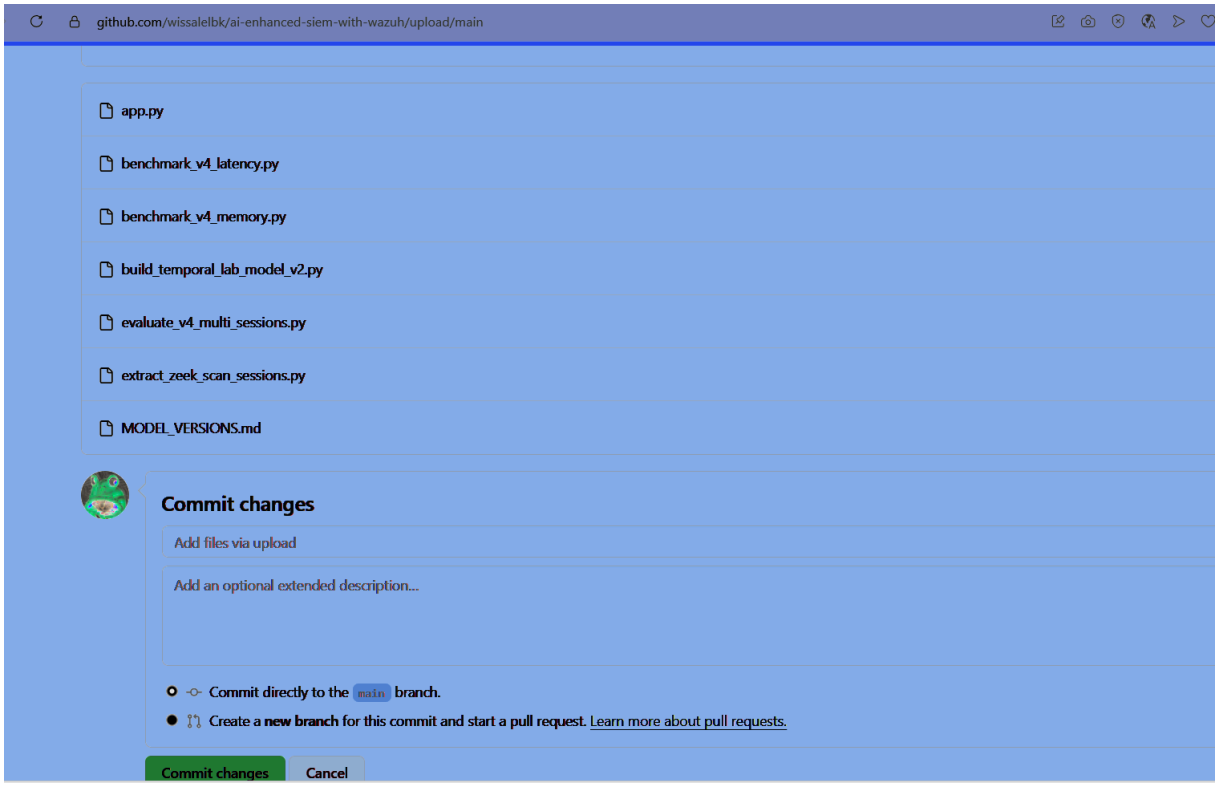


Figure 3.13: Training/evaluation scripts and `MODEL_VERSIONS.md` committed to the project's GitHub repository, formalising model version control

General Conclusion

Significance of the Topic. Centralizing and analyzing security event logs in real time is no longer optional for organizations with a growing, heterogeneous attack surface. This internship addressed that need directly for YaneCode Digital, delivering a Wazuh-based SIEM foundation while exploring how far machine learning could realistically push detection beyond fixed rules.

Results. The deterministic core is a clear success: four Sigma-derived detection rules – brute force, privilege escalation, network scan, and data exfiltration – were built, deployed, and each confirmed under real attack conditions on the lab. **This layer is validated and operational, and covers all four required attack categories.** The machine-learning layer took a longer path to a narrower outcome: two early modelling attempts (Isolation Forest, then Random Forest/XGBoost) and a rebalanced fusion (v3) each failed to generalise to live traffic, in ways that progressively narrowed the diagnosis – from a 30:1 dataset-fusion imbalance to a deeper, structural issue: classifying connections one at a time cannot represent a scan, which is defined by repetition across many connections from one source. Adding behavioural features aggregated per source IP over a sliding time window (v4) resolved this, **for network scan specifically**: the Random Forest model reached 99.94% recall at 0.09% false positives on the initial Colab training run, on a scan captured after training and never seen in any form. This model was then extracted and exposed through a standalone Streamlit prediction interface deployed on the lab's SIEM server, at the supervisor's direction, to move the project from detection to prediction. A critical deployment-time bug – a reversion to an earlier, incomplete labelling method on the server's own copy of the training script – was subsequently found and corrected, with the fix re-validated on the same out-of-sample scan at **99.78% recall / 0.06% false positives**. This server-trained, bug-fixed model is the one actually running behind the deployed Streamlit prototype, and its 99.78%/0.06% result is the **official final result** for this project, read together with the independent 2 September multi-session test (Section 3.3.6), which also surfaced one documented weakness (50.72% false-positive rate on a slow, low-rate scan against broadcast noise). **Brute force, privilege escalation, and data exfiltration are not covered by this model and remain detected exclusively by the deterministic Wazuh/Sigma rules.** The model was subsequently re-evaluated on five further scan sessions and a second, previously unseen attacker IP, confirming it generalises beyond a single attacker's profile, and its operational performance (latency, memory, restart behaviour, model versioning) was measured directly on the SIEM Server with no bottleneck identified. **The deterministic layer is validated and operational; the machine-learning layer is a validated prototype, deployed and tested in the lab, standing alongside Wazuh rather than integrated into it.**

Conclusion and Outlook. With both layers now operational, the remaining work is narrower than at any earlier point in the project: extend the same temporal-feature approach to data exfiltration, the other attack category with genuine volumetric ambiguity; and evaluate direct integration of the prediction model into the Wazuh alerting pipeline as an alternative, more tightly coupled path to the current standalone Streamlit interface. This iterative, diagnosis-driven methodology – testing hypotheses with isolated experiments, tracing each failure to its root cause rather than assuming a single fix, and re-validating rigorously after every correction, including a deployment-time regression – is itself the main technical outcome of the machine-learning phase carried out this summer.

Appendices

Appendix A – Detailed Next Steps

- **Complete the CICIDS2017 data-leakage audit (Section 3.3.3) – partially done.** Check (2), duplicate/label-conflict detection, has been run on the raw CICIDS2017 corpus and found a real (small) contamination signal (719 feature-identical groups with conflicting labels). Checks (1) (train/test split ordering relative to oversampling) and (3) (no engineered feature trivially encodes the label) have not yet been run.
- **Extend temporal features to data exfiltration.** The same sliding-window, per-source-IP approach that resolved network-scan detection is a natural fit for exfiltration, the other attack category with genuine volumetric ambiguity (large or repeated transfers rather than a single connection's byte count).
- **Evaluate operational performance – done (Section 3.3.7).** Throughput, latency, memory consumption, and restart behaviour were measured directly on the SIEM Server; no bottleneck was identified. Handling of missing/incomplete fields and behaviour under sustained, multi-hour load remain to be measured.
- **Evaluate direct Wazuh integration** as an alternative to the current standalone Streamlit interface, embedding the prediction step directly into the alerting pipeline rather than as a separate lookup tool.
- **Formalise the deployment process – partially done (Section 3.3.7).** Training/evaluation scripts and a model-version history (`MODEL_VERSIONS.md`) are now published on the project's GitHub repository. Binary model artifacts (`.pkl`) are still not version-controlled (kept only on the SIEM Server); adopting a dedicated tool (MLflow, DVC, or Git LFS) for these remains an open item.
- **Add remaining methodology screenshots.** Wazuh dashboard/agent screenshots (Section 3.3.1) have been added. Screenshots of the dataset-construction and training scripts (`fetch_alerts.py`, `merge_datasets.py`, `train_isolation_forest.py` output, and the Colab 1 vs Colab 2 diagnostic) are still to be captured, inserted, captioned, and referenced in Section 3.3.3.

Appendix B – Identified Limitations

- Brute force and privilege escalation could not be modelled with machine learning at all, due to an extreme shortage of positive examples (2 and 1, respectively, out of 38 total Windows authentication events); these remain covered exclusively by deterministic rules.

- The prediction interface currently covers network scan only; brute force, privilege escalation, and exfiltration remain covered exclusively by the deterministic Wazuh/Sigma rules.

Bibliography

Academic references

1. I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, Funchal, Madeira, Portugal, 2018, pp. 108–116.

2. F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.

3. N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

4. L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

5. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 785–794.

6. S. Staniford, J. A. Hoagland, and J. M. McAlerney, “Practical Automated Detection of Stealthy Portscans,” *Journal of Computer Security*, vol. 10, no. 1–2, pp. 105–136, 2002.

Online resources *(consulted 29 August 2026 unless noted otherwise)*

7. Wazuh Documentation – `https://documentation.wazuh.com`

8. Sigma Project (SigmaHQ) – `https://github.com/SigmaHQ/sigma`

9. Suricata Documentation – `https://docs.suricata.io`

10. Zeek Documentation – `https://docs.zeek.org`

11. MITRE ATT&CK Framework – `https://attack.mitre.org`

12. CICIDS2017 Dataset landing page (Canadian Institute for Cybersecurity) – `https://www.unb.ca/cic/datasets/ids-2017.html`

13. scikit-learn Documentation – `https://scikit-learn.org`

14. XGBoost Documentation – `https://xgboost.readthedocs.io`

15. imbalanced-learn (SMOTE implementation) Documentation – https://imbalanced-learn.org

16. OpenSearch Documentation – https://opensearch.org/docs

17. CICFlowMeter – https://www.unb.ca/cic/research/applications.html

18. YaneCode Digital – https://yanecode.org